

PRD: Aster Roadside Assist

Vision and Goals

Aster Roadside Assist is an AI voice agent for the first minutes of a roadside assistance claim. It confirms safety, verifies identity, captures vehicle, location, and incident details, then gives the customer a clear next step without repeated handoffs.

The goal is better customer reassurance with tighter operating control: faster intake, more complete first-contact data, fewer manual coverage checks, fewer avoidable dispatch errors, lower cost-to-serve, and lower cost of revenue per roadside case. Human agents stay focused on complex, high-risk, or low-confidence situations.

The prototype targets the areas insurers will worry about before trusting automation: safety, vague locations, noisy incident descriptions, identity verification, privacy, coverage confidence, and human oversight. Success is measured by customer satisfaction, data completeness, decision accuracy, escalation quality, auditability, reduced dispatch waste, and lower claim leakage from bad first-contact data.

Prioritized Features in the Prototype

The prototype targets the objections a skeptical insurer will raise: sloppy outputs, hallucinated decisions, privacy mistakes, and unclear accountability. The design answer is simple: the LLM handles conversation, while a deterministic backend owns workflow state, validation, policy checks, and outcomes.

1. **Will the agent miss safety issues or create incomplete cases?** The voice intake starts with safety, then collects identity, vehicle, location, and incident step by step. The backend state machine decides whether required facts are complete before coverage or next-best-action can run.
2. **Will the LLM improvise instead of following insurance rules?** The Kotlin/Spring Boot backend owns validation gates, claim state, terminal outcomes, transcripts, and tool-call trace. The LLM can converse, but it cannot skip workflow requirements or silently decide coverage.
3. **Will it fail on vague roadside locations?** The demo uses backend geoparsing with Google Maps links. If the first location is ambiguous, the agent asks a follow-up and the backend stitches multiple caller clues into a dispatchable place. This is not production-complete, but it proves the agent can handle the kind of imperfect location descriptions callers actually give.
4. **Will it respect privacy?** Customer and vehicle details are withheld from the conversational model until verification succeeds. Known callers provide selected PIN digits; unknown callers provide name, birthdate, and PIN digits. Failed verification stops intake without data disclosure.
5. **Will decisions be explainable to customers and agents?** A structured-output model classifies the incident into controlled types; backend policy logic maps it to covered, not covered, repair truck, tow truck, or human callback. The customer sees a fake SMS outcome.
6. **Will humans trust and audit the system?** The observation UI shows case status, extracted facts, validation gates, transcript/tool trace, outcome, and KPIs so supervisors can look under the hood.
7. **What happens when automation is wrong or uncomfortable?** The caller can ask for a human, and the system routes specialist, non-policyholder, or low-confidence cases to callback.

Milestones

Week 1: Workflow mapping and real data contracts. Deeply map how human agents operate today: questions asked, systems checked, policy decision points, escalation rules, and dispatch handoffs. In parallel, secure API contracts for customer, vehicle, policy, and entitlement data so the production state machine is based on real workflow and real records, not assumptions.

Weeks 2-3: Implementation. Two FTEs run in parallel. Track A owns the customer-facing path: phone integration, voice behavior, SMS wording, and observation UI. Track B owns the control plane: deterministic state machine, identity verification, real data integration, policy checks, transcript/tool-call persistence, and audit trail. Set up offline evals against historical cases and real transcripts, including STT/TTS/LLM provider benchmarks for accuracy, latency, tone, and control.

Week 4 - Track A: Controlled shadow pilot. Run a test or mirrored phone line with insurer QA/ops specialists and field-side reviewers. Human agents remain responsible for outcomes while online evals compare AI case completeness, location resolution, policy decisions, escalation quality, STT entity accuracy, tone, and latency against the current workflow. Continue replaying human-agent transcripts offline to track accuracy improvements.

Weeks 4+ - Track B: Dispatch and readiness hardening. Add provider availability, ETA ranking, SMS delivery, dispatch handoff simulation, callback queues, and supervisor metrics. Lock thresholds for safety, auth, location, incident classification, coverage, escalation, and voice quality.

Technical Risks and Open Questions

Location resolution. Google Maps works well for many landmarks and road descriptions, but roadside cases may require specialized road-network data, highway marker handling, "between junctions" parsing, and provider coverage zones. The prototype proves potential, not production completeness.

Voice reliability. The current OpenAI Realtime setup is fast to prototype and convincing for a demo, but background noise, VAD/noise tuning, interruption handling, call-ending behavior, roadside-specific STT accuracy, and TTS control need work. Road names, junctions, landmarks, number plates, PIN digits, and vehicle terms should be measured explicitly. For TTS, production needs adequate control over pacing, pronunciation, pauses, and tone. We can evaluate specialist voice providers, but keep orchestration, state, tool policy, audit trail, and provider routing under our control.

Policy and workflow. For production, we need a much clearer joint understanding of the client's current roadside workflow: what human agents ask, what systems they check, which decisions are rules vs. judgement calls, when they escalate, and how dispatch partners are selected. Only after that should we encode policy rules, state-machine steps, and human handoff points. The LLM should classify and extract facts, not silently invent coverage rules or operational process.

Privacy and security. Production needs stronger authentication choices, consent and recording disclosures, PII minimization, retention rules, fraud monitoring, and full audit logging.

AI Integration Approach

The system separates conversational AI from business authority. The prototype uses OpenAI Realtime (gpt-realtime-2, voice marin) for fast voice iteration, and a small structured-output model (gpt-5.4-nano) to classify incident descriptions into controlled enums. The backend state machine determines the next required fact and whether the case can proceed. A backend geoparsing service resolves spoken locations into dispatchable places. Coverage and next-best-action decisions are backend-owned and traceable.

The improvement loop is eval-driven. Every call should produce audio, transcript, corrected transcript, extracted facts, tool trace, decision path, and verified outcome. Offline evals replay this data against classifiers, prompts, state-machine changes, location resolution, and STT providers before release. Online evals run in shadow mode with QA labels for safety, identity, location, incident, coverage, escalation, tone, and latency. The voice-intake LLM is a clear post-training candidate once we have real transcripts and labels: we can fine-tune it to ask better follow-ups and extract incidents more reliably, while the backend still owns state and policy decisions. Roadside STT should improve through provider benchmarks, domain vocabulary, and custom speech models if entity-level errors remain high. Changes promote only when predefined thresholds improve. If voice reliability becomes the bottleneck, we would keep orchestration in-house and swap STT, LLM, and TTS providers underneath it.